# Ejercicio 04: clasificador visual con CNN y Model Card

INF-8239 Ciencia de Datos II, Unidad 02 (U02.E04)

| | |
|---|---|
| Estudiante | Roque Alexis Ventura Vallejo |
| Matrícula | 100805873 |
| Repositorio | github.com/RogueVentura/UASD-Maestria-Ciencia-De-Datos, Ciencia de datos II/Unidad II/Ejercicio 04 |

## 1. Dataset y problema

Fashion-MNIST (Xiao et al., 2017; licencia MIT) tiene 70,000 imágenes de prendas en escala de grises de 28×28 píxeles y 10 clases: 60,000 de entrenamiento y 10,000 de prueba. TensorFlow lo descarga a su caché. El objetivo es comparar, con la misma partición, un baseline denso y una CNN, y decidir si la mejora predictiva justifica el costo computacional.

## 2. Auditoría y partición reproducible

La partición es fija: las primeras 54,000 imágenes de entrenamiento entrenan, las últimas 6,000 validan (EarlyStopping y checkpoint) y las 10,000 de prueba solo se usan al final. *audit_splits* compara un hash MD5 de los píxeles para encontrar duplicados exactos dentro de cada partición y entre particiones (*reports/audit.json*).

| Partición | Forma | Rango | Imágenes por clase | Duplicados | Compartidas |
|---|---|---|---|---|---|
| Entrenamiento | 54,000 × 28 × 28 × 1 | [0, 1] | 5,367 a 5,445 | 0 | con validación: 0 |
| Validación | 6,000 × 28 × 28 × 1 | [0, 1] | 555 a 633 | 0 | con prueba: 0 |
| Prueba | 10,000 × 28 × 28 × 1 | [0, 1] | 1,000 exactas | 0 | con entrenamiento: 0 |

No hay fuga por imágenes repetidas y la prueba está balanceada. La validación no es estratificada (555 a 633 por clase), lo que no afecta la elección del mejor punto. *validate_images* rechaza formas, rangos o etiquetas fuera de contrato antes de entrenar. Con semilla 42 y TF_DETERMINISTIC_OPS=1, todas las corridas en el mismo equipo dieron el mismo F1 hasta el último decimal.

## 3. Baseline y CNN

Los tres modelos usan Adam, entropía cruzada, lotes de 128, máximo 8 épocas y EarlyStopping (paciencia 2, restaura los mejores pesos según la pérdida de validación). La CNN Flatten conserva los filtros de la CNN del laboratorio, pero reemplaza GlobalAveragePooling por Flatten para probar si la pérdida de posición era lo que la limitaba. Su presupuesto de parámetros es casi igual al del baseline, así que la comparación es justa.

| Modelo | Arquitectura | Parámetros |
|---|---|---|
| Baseline denso | Flatten, Dense(64, ReLU), Dense(10, softmax) | 50,890 |
| CNN del laboratorio | Conv2D(32, 3×3), MaxPool, Conv2D(64, 3×3), MaxPool, GlobalAveragePooling, Dropout(0.25), Dense(10) | 19,466 |
| CNN Flatten | Mismas convoluciones y pooling, Flatten, Dropout(0.25), Dense(10) | 50,186 |

## 4. Métricas, curvas y errores por clase

| Modelo (prueba, 10,000 imágenes) | F1-macro 8 ép. | Accuracy 8 ép. | F1-macro 30 ép. | Mejor época (30) |
|---|---|---|---|---|
| Baseline denso | 0.864 | 0.865 | 0.875 | 17 de 19 |
| CNN del laboratorio | 0.789 | 0.793 | 0.848 | 30 de 30 |
| CNN Flatten | 0.900 | 0.902 | 0.910 | 13 de 15 |

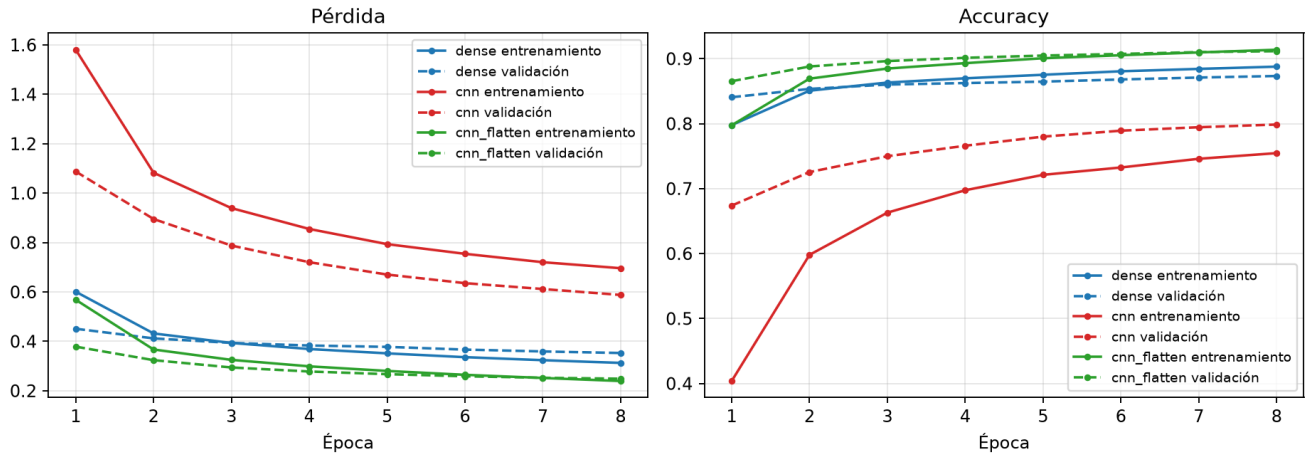
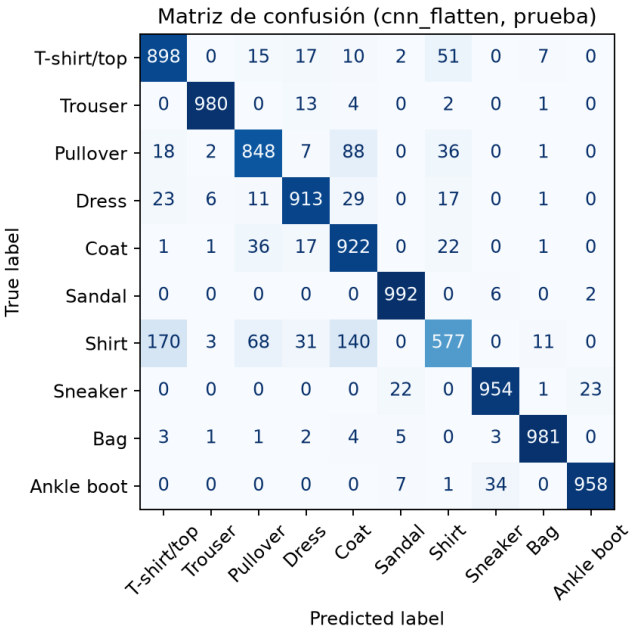



Figura 1. Pérdida y accuracy por época (continua: entrenamiento; discontinua: validación), 8 épocas.

La CNN del laboratorio subajusta: sus dos pérdidas siguen bajando en la época 8 y la validación supera al entrenamiento porque el Dropout solo actúa al entrenar. Con los mismos filtros, la CNN Flatten aprende desde la primera época y termina con entrenamiento y validación casi iguales, sin sobreajuste. El baseline denso se estanca antes. Con 30 épocas el orden no cambia: 0.910 (CNN Flatten), 0.875 (denso) y 0.848 (CNN del laboratorio). Promediar la posición de los rasgos era lo que limitaba a la CNN original. Con 30 épocas, EarlyStopping detiene la CNN Flatten en la época 15: la validación deja de mejorar desde la 13, primer indicio de sobreajuste, y se conservan esos pesos.




| Clase | Denso | CNN lab | CNN Fl. | Recall Fl. |
|---|---|---|---|---|
| T-shirt/top | 0.82 | 0.72 | 0.85 | 0.90 |
| Trouser | 0.97 | 0.96 | 0.98 | 0.98 |
| Pullover | 0.75 | 0.67 | 0.86 | 0.85 |
| Dress | 0.86 | 0.80 | 0.91 | 0.91 |
| Coat | 0.78 | 0.66 | 0.84 | 0.92 |
| Sandal | 0.95 | 0.92 | 0.98 | 0.99 |
| Shirt | 0.65 | 0.42 | 0.68 | 0.58 |
| Sneaker | 0.93 | 0.90 | 0.96 | 0.95 |
| Bag | 0.96 | 0.93 | 0.98 | 0.98 |
| Ankle boot | 0.95 | 0.91 | 0.97 | 0.96 |

Figura 2. Matriz de confusión de la CNN Flatten en prueba (filas: clase real). A la derecha, F1 por clase de los tres modelos y recall de la CNN Flatten.

Shirt es la clase más difícil en los tres modelos: la CNN Flatten recupera el 58 % de las camisas (F1 0.68), frente al 36 % de la CNN del laboratorio. Sus confusiones principales son Shirt con T-shirt/top (170), Shirt con Coat (140) y Pullover con Coat (88): prendas de torso con la misma silueta a 28×28. Trouser, Sandal, Sneaker, Bag y Ankle boot superan 0.95 de F1.

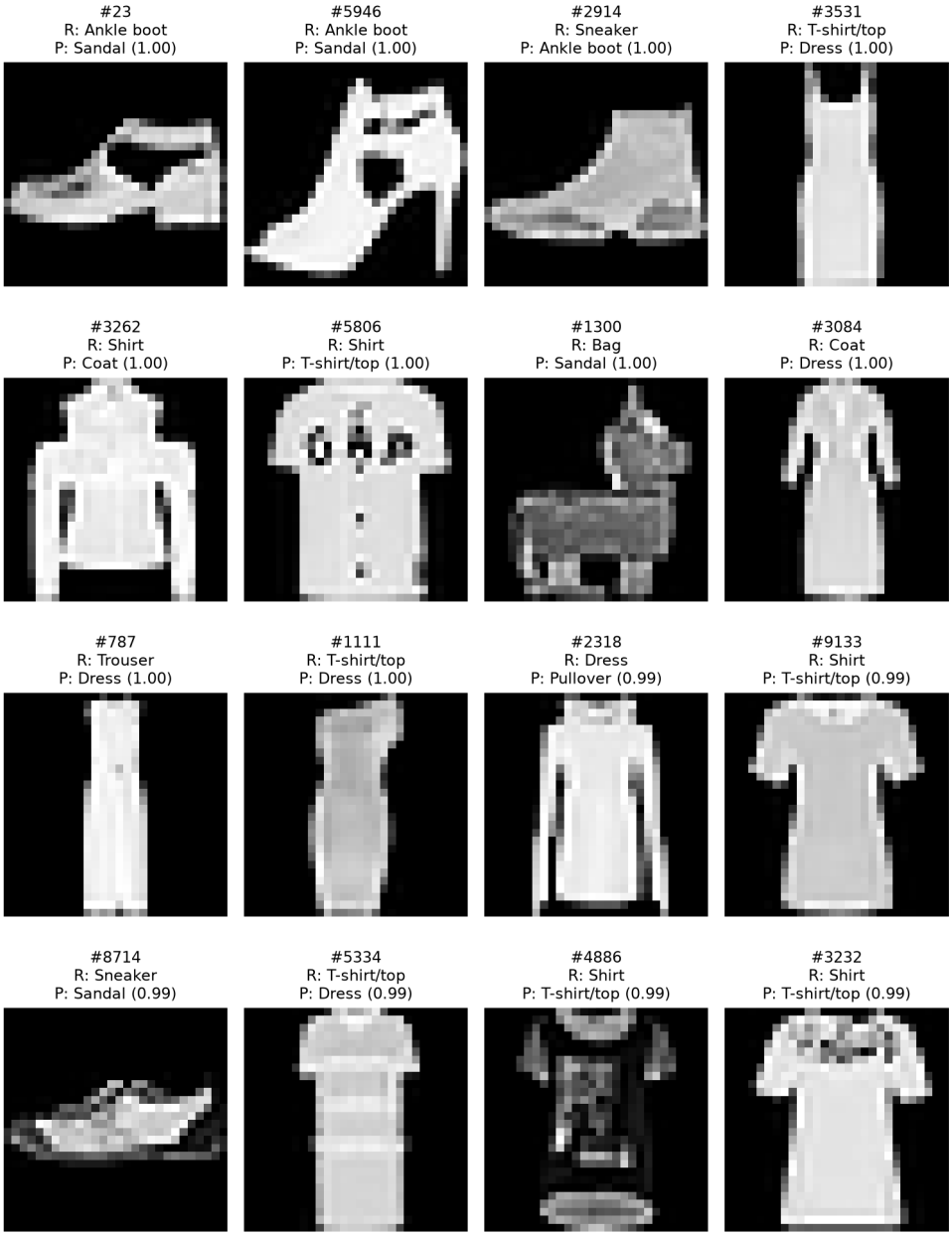



Figura 3. Los 16 errores de la CNN Flatten con mayor confianza (#índice de prueba, R: real, P: predicha y su probabilidad). La figura equivalente de la CNN del laboratorio está en reports/cnn_errors.png.

En los errores de mayor confianza hay tres grupos: (1) calzado con rasgos mezclados, como botines con aberturas predichos como Sandal (#23, #5946) o un sneaker de caña alta predicho como Ankle boot (#2914); (2) camisas de manga corta predichas como T-shirt/top (#5806, #9133, #3232); y (3) imágenes ambiguas o con etiqueta dudosa, como el #1300 (etiquetado Bag, con forma de figura de animal) y el #787 (etiquetado Trouser, con silueta de vestido). Varios errores tienen probabilidad de 1.00: la salida softmax no está calibrada y no debe leerse como certeza.

## 5. Costo, tamaño y decisión técnica

| Medida (8 épocas, CPU) | Baseline denso | CNN del laboratorio | CNN Flatten |
|---|---|---|---|
| F1-macro en prueba | 0.864 | 0.789 | 0.900 |
| Parámetros | 50,890 | 19,466 | 50,186 |
| Archivo .keras | 619 KB | 262 KB | 622 KB |
| Tiempo de entrenamiento | 6.4 s | 66.8 s | 108.6 s |
| Inferencia (mediana de 5 repeticiones) | 0.010 ms/img | 0.060 ms/img | 0.062 ms/img |

Hardware: Intel Core i7-1165G7 (2.8 GHz), 15.7 GB de RAM, sin GPU, Windows 11; Python 3.12.13, TensorFlow 2.21.0 (CPU), Keras 3.15.1, scikit-learn 1.9.1. Inferencia sobre las 10,000 imágenes de prueba, lotes de 256, tras una pasada de calentamiento. Los tiempos varían entre corridas: el denso fue el más rápido en todas las mediciones, pero la relación con la CNN Flatten fue de 1.3 a 6 veces (script de 8 y 30 épocas y notebook).

**Decisión.** Elijo la CNN Flatten. Con prácticamente los mismos parámetros y el mismo tamaño de archivo que el baseline, gana 3.6 puntos de F1-macro y mejora todas las clases, sobre todo las de torso. El costo es real: entrena 17 veces más lento (109 s frente a 6 s) e infiere entre 1.3 y 6 veces más lento según la medición. En términos absolutos, 0.06 ms por imagen en CPU sigue permitiendo miles de imágenes por segundo, y el entrenamiento cabe en un par de minutos, así que la mejora justifica el costo para este benchmark. Si la latencia en CPU fuera la restricción principal, el baseline denso sería la alternativa. La CNN del laboratorio se descarta: cuesta más que el denso y rinde menos. Menos parámetros no significa menor costo, porque las convoluciones hacen muchas más operaciones por imagen.

## 6. Código, reproducibilidad y repositorio

| Ruta | Contenido |
|---|---|
| src/inf8239_u02_cv/ | data.py (clases, normalización, validación, partición, auditoría), models.py (build_dense, build_cnn, build_cnn_flatten), evaluation.py (inferencia repetida, confusiones, tamaño) |
| scripts/train_cv.py | Entrena los tres modelos y genera métricas, figuras y modelos |
| notebooks/01_fashion_mnist_cnn.ipynb | Notebook ejecutado: auditoría, recarga de modelos y recálculo del F1 (coincide con el reporte), curvas, errores y costo |
| reports/, models/ | Métricas JSON, figuras, este informe y los modelos .keras (8 y 30 épocas) |
| README.md, MODEL_CARD.md, requirements.txt, uv.lock | Instrucciones, ficha del modelo y entorno fijado |

```
uv sync --extra cpu
uv run python scripts/train_cv.py --epochs 8
uv run python scripts/train_cv.py --epochs 30 --run-name epochs30
uv run pytest -q
uv run jupyter nbconvert --to notebook --execute --inplace notebooks/01_fashion_mnist_cnn.ipynb
```

Comprobé la ejecución desde un clon limpio del repositorio siguiendo el README: el entorno se crea y las pruebas pasan. El historial de Git conserva los commits del laboratorio (*chore: initialize isolated computer vision project*, *feat: compare dense baseline and CNN*) y los posteriores.

## 7. Pruebas automatizadas

| Archivo | Qué comprueba |
|---|---|
| test_data.py (8) | Normalización a (n, 28, 28, 1) en [0, 1]; rechazo de formas, rangos y etiquetas inválidas; 10 clases; partición fija y disjunta; detección de imágenes compartidas; contrato del dataset real (54,000 / 6,000 / 10,000, forma, rango y 10 clases) |
| test_model_contract.py (8) | Salida (n, 10) con probabilidades finitas, no negativas y que suman 1 en los tres modelos; contrato original del laboratorio; presupuesto de parámetros; contrato de los tres modelos guardados en models/ |

Resultado: 16 passed con *uv run pytest -q*; ruff sin errores.

## 8. Model Card

*MODEL_CARD.md* documenta el modelo elegido (CNN Flatten): uso previsto académico; fuera de alcance fotos reales, imágenes médicas, OCR, vigilancia o decisiones sobre personas; dataset, particiones y preprocesamiento; métricas globales y por clase; hardware, tiempos y versiones; riesgos (Shirt, errores con confianza alta, etiquetas dudosas, una sola semilla) y supervisión humana antes de cambiar de dominio.

## 9. Conclusión

| Pregunta del cierre | Respuesta |
|---|---|
| Resultado principal | La CNN Flatten obtiene F1-macro 0.900, frente a 0.864 del baseline y 0.789 de la CNN del laboratorio, con la misma partición. |
| Evidencia predictiva | Mejora en las 10 clases y se mantiene con 30 épocas. Las curvas muestran que la CNN del laboratorio subajustaba por su cabeza GlobalAveragePooling. |
| Clase más difícil | Shirt (recall 0.58 en la CNN Flatten), confundida con T-shirt/top y Coat. |
| Costo comparado | Mismos parámetros y tamaño que el baseline, pero 17 veces más tiempo de entrenamiento y entre 1.3 y 6 veces más tiempo de inferencia en CPU. |
| ¿La mejora justifica el costo? | Sí para este benchmark: 3.6 puntos de F1 por un costo absoluto bajo. No si la latencia en CPU fuera crítica. |
| Limitación del benchmark | Imágenes centradas, 28×28, en grises y fondo limpio; una sola semilla; etiquetas dudosas. No se transfiere a otros dominios. |
| Decisión antes de usar otro dominio | Recolectar y etiquetar datos propios, repetir auditoría y partición, reentrenar y evaluar por clase con revisión humana. |

## 10. Uso de inteligencia artificial

| | |
|---|---|
| Herramienta | Claude Code (Claude Opus 5.5, Anthropic) en Visual Studio Code, octubre de 2026. |
| Mi participación | Preparé el entorno con uv, ejecuté el entrenamiento inicial y las pruebas del proyecto base, e hice el commit inicial. Verifiqué el informe y pedí correcciones que se aplicaron, como guardarlo en reports/. |
| Prompts (resumen) | Revisar el proyecto, luego de haber corrido yo los scripts del laboratorio. Pedí compararlo con el mandato y la rúbrica, apoyarme en lo que faltara y explicarme cada punto; completar la evidencia faltante con resultados reales. |
| Tareas asistidas | Ampliar train_cv.py, crear evaluation.py, la CNN Flatten, el notebook y 13 pruebas nuevas, y proponer el README, el Model Card y el borrador del informe. |
| Verificación | Las cifras salen de reports/*.json. Todas las corridas dieron el mismo F1; el notebook lo recalcula con los modelos guardados. Las pruebas pasan desde un clon limpio. |
| Correcciones | Se corrigió el dato de una prueba que fallaba; se dejó de excluir reports/ y models/; la CNN del laboratorio rendía menos que el denso y, en lugar de aceptar esa conclusión, se probó la causa con la CNN Flatten; se agrandaron los rótulos de la figura de errores; el informe se movió a reports/. |
| Responsabilidad | Asumo la responsabilidad de lo entregado. |

## 11. Fuentes


Abadi, M., et al. (2016). *TensorFlow: Large-scale machine learning on heterogeneous distributed systems*. arXiv. https://doi.org/10.48550/arXiv.1603.04467

Mitchell, M., et al. (2019). Model cards for model reporting. *Proceedings of FAT* 2019*, 220-229. https://doi.org/10.1145/3287560.3287596

Pedregosa, F., et al. (2012). *Scikit-learn: Machine learning in Python*. arXiv. https://doi.org/10.48550/arXiv.1201.0490

Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms*. arXiv. https://doi.org/10.48550/arXiv.1708.07747